Class06: R functions

Julissa Gonzalez (A18495188)

Table of contents

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the bracket when we call the function),
- the *body* (the lines of R code that do the work of the function).

A first function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either "required" or "optional. The later have fall-back defualt values that will be used if the user does not specify them.

```
add<- function(...){
  return(sum(... ))
}
```

can wew use our new function:

```
add(10,100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Q1A. Your first version of the function should add two input numbers together. For example, add(4,7) should return 11

```
add(4, 7)
```

```
[1] 11
```

Q1B. For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10))

```
add(4, 7)
```

```
[1] 11
```

```
add( c(4,7,10) )
```

```
[1] 21
```

Q1C. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return

```
add(1, 2, 3, -4)
```

```
[1] 2
```

```
ans<- add(1,2,3)
ans
```

```
[1] 6
```

We can explicitly set a **return** value output from a function (rather that the default last time)

```
add<- function(x,y=0,z=0){
  return(sum(x,y,z))
  cat("Is it break time yet?\n")
}

add(10,100)
```

```
[1] 110
```

can we use our new function:

A `generate_dna()` function

a useful function here is the "base R" `sample()` function:

```
sample(1:5,size=60, replace=TRUE)
```

```
 [1] 1 1 1 4 1 1 1 2 1 1 5 4 1 3 3 3 4 2 4 1 4 4 3 2 4 5 1 1 4 4 1 3 4 4 3 5 2 5
[39] 4 4 4 1 1 1 4 1 3 3 4 2 5 5 2 2 1 3 5 3 2 4
```

We can use this to make a random nucleotide sequwnce if we draw from "A","C", "G", and "T"...

```
sample(x=c("A","C","G","T"), size=10, replace=TRUE)
```

```
 [1] "C" "C" "T" "T" "G" "T" "C" "T" "A" "A"
```

> **Q2.** Write a function generate_dna() that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A", "C". [1 pt]

```
generate_dna <- function(len=10) {
  sample(x=c("A","C","G","T"), size=len, replace=TRUE)
}
```

```
generate_dna()
```

```
 [1] "C" "G" "C" "T" "T" "T" "T" "A" "G" "G"
```

```
generate_dna(100)
```

```
  [1] "A" "A" "C" "A" "A" "G" "G" "A" "G" "T" "C" "C" "A" "T" "G" "C" "T" "C"
 [19] "C" "G" "G" "A" "C" "T" "C" "C" "A" "T" "T" "C" "T" "A" "C" "A" "A" "T"
 [37] "G" "C" "A" "C" "A" "T" "A" "T" "T" "G" "C" "C" "T" "C" "G" "G" "A" "C"
 [55] "T" "G" "A" "A" "G" "A" "C" "A" "A" "G" "C" "A" "G" "G" "G" "C" "T" "A"
 [73] "T" "T" "T" "C" "G" "C" "A" "T" "A" "T" "C" "C" "G" "C" "A" "A" "A" "T"
 [91] "T" "C" "T" "A" "T" "G" "C" "C" "A" "C"
```

Q2b:Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a singe vector of multiple letters). For example "AAGGCTG". [1 pt]

```
generate_dna <- function(len=10,single.element=TRUE) {
  ans<-sample(c("A","C","G","T"), size=len, replace=TRUE)
  #cat("Hello...")
  if(single.element){
    #cat("Is it me your looking for...")
   ans<- paste(ans, collapse= "")
  }

  ##Format as FASTA with an ID line
  cat(paste(">len",len,"\n", sep=""))
  cat(ans)
  cat("\n")

  ##
  ##
  return(ans)
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()`and `return()`

```
generate_dna(44)
```

```
>len44
GTTTTCCAACCACTCTGGGTACGCAGGTGTGTAAAGCGCAGCTT
```

```
[1] "GTTTTCCAACCACTCTGGGTACGCAGGTGTGTAAAGCGCAGCTT"
```

```
generate_dna(single.element = TRUE)
```

```
>len10
CTTGTTTCTT
```

```
[1] "CTTGTTTCTT"
```

```
generate_dna(single.element=FALSE)
```

```
>len10
T A T C A T A A G G
```

```
 [1] "T" "A" "T" "C" "A" "T" "A" "A" "G" "G"
```

```
paste(c("A","C","G"),collapse="---")
```

```
[1] "A---C---G"
```

Q2C.Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example:

```
>len9
CGAAGGCTG
```

```
cat("hello \n there")
```

```
hello
 there
```

```
generate_dna()
```

```
>len10
TGAGGGTCTC
```

```
[1] "TGAGGGTCTC"
```

Write a `generate protien()` function

> **Q3.** Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG".

```
generate_protien<-function(len=9){
  aa<-c("A","R","N","D","C","E","Q","G","H","I","L","K","K","M","F","P","S","T","W","Y","V")
  ans<-sample(x=aa, size=len, replace=TRUE)
  paste(ans,collapse="")
  }
```

```
generate_protien(6)
```

```
[1] "AKAKNP"
```

Write a 'Generate random protien sequences of length 6 to 13

> **Q4.**Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand.

```
for(l in 6:13){
  cat(">", l, "\n", sep="")
  cat(generate_protien(l), "\n")
}
```

```
>6
ILYSRE
>7
QNGKIML
>8
GMFRFHGV
>9
ATLKCGKRR
>10
HWINIEKITA
>11
LRYQMGHKKYT
>12
KAYSKVEYSYHV
>13
NGVYRSPLKIWGQ
```

```
generate_protien(6)
```

```
[1] "KWNYGQ"
```

```
generate_protien(7)
```

```
[1] "WSVCTKK"
```

```
generate_protien(8)
```

```
[1] "ENATTWQN"
```

```
generate_protien(9)
```

```
[1] "CCRDFDRIF"
```

Are Our peptides

> **Q5**:We will define a peptide as "unique in nature" (in the specific sense used in this lab) if no match exists with 100% coverage AND 100% identity to a sequence already in nr. Note that this is a conservative definition — a short exact match may still exist as a sub-string of a longer protein, and will typically be reported by BLASTp with 100% identity but less than 100% coverage of the subject (which we are not using here).

Length	Ide	Cov	Unique
6	100	100	N
7	100	100	N
8	100	100	N
9	100	88	Y
10	100	80	Y
11	88	82	Y
12	100	67	Y
13	88	69	Y

Connecting your findings to immunology (MHC class II and T-cell activation)

> **Q6.** In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

Based Q5 results the minimum peptide length is around 9 amino acids, since shorter peptides 6-8 were not unique. Peptides of length 9 and above began to show uniqueness. This means that shorter sequences are more likely to occur by chance and match existing proteins. Short peptides would be a poor choice for the immune system because they may not be specific enough to distinguish between self and non-self proteins. This could increase the risk of false recognition or failure to detect pathogens. Longer peptides provide greater specificity, improving accurate immune responses